- *Azure Web App*—In a special synced folder, shared between regions
- *IIS without user profile*—Encrypted in the registry
- *Account with user profile*—In %LOCALAPPDATA%\ASP.NET\DataProtection-Keys on Windows, or ~/.aspnet/DataProtection-Keys on Linux or macOS
- *All other cases*—In memory; when the app restarts, the keys will be lost

So why do you care? For your app to be able to read your users' authentication cookies, it must decrypt them by using the same key that was used to encrypt them. If you're running in a web-farm scenario, by default each server has its own key and won't be able to read cookies encrypted by other servers.

To get around this, you must configure your app to store its data protection keys in a central location. This could be a shared folder on a hard drive, a Redis instance, or an Azure blob storage instance, for example.

Microsoft's documentation on the data protection APIs is extremely detailed, but it can be overwhelming. I recommend reading the section on configuring data protection, ("Configure ASP.NET Core Data Protection," http://mng.bz/d40i) and configuring a key storage provider for use in a web-farm scenario ("Key storage providers in ASP.NET Core," http://mng.bz/5pW6). I also have an introduction to the data protection APIs on my blog at http://mng.bz/yQQd.

It's worth clarifying that the CSRF vulnerability discussed in this section requires that a malicious site does a full form `POST` to your app. The malicious site can't make the request to your API using client-side-only JavaScript, as browsers block JavaScript requests to your API that are from a different origin.

This is a safety feature, but it can often cause you problems. If you're building a client-side SPA, or even if you have a little JavaScript on an otherwise server-side rendered app, you may need to make such cross-origin requests. In the next section I describe a common scenario you're likely to run into and show how you can modify your apps to work around Pit.